

PRAXIS (CCE e-Lab) - Comprehensive Technical Architecture & Internals

This document is a deep-dive technical reference for the PRAXIS (E-Lab) platform. It covers the architecture, the technology stack and its contributions, and the inner workings of core modules such as the execution sandbox, AI Hint system, and RAG Question generator.

1. Executive Overview

PRAXIS is a self-hosted, high-concurrency coding and placement skill development platform designed for the Department of Computer and Communication Engineering (CCE) at NMAM Institute of Technology. It handles coding practice, proctored assessments, AI tutoring, and automated certification for up to 400 concurrent students without requiring external internet APIs.

2. Technology Stack & Component Contributions

2.1 Backend Core

- **Django 5.0 (Python 3):** The core web framework. Handles routing, authentication, business logic, templating, and ORM mapping.
- **Django REST Framework (DRF):** Used for providing APIs to the frontend editor (submissions, hint unlocking).
- **PostgreSQL 15 (e1ab-db):** The primary relational database. Stores users, courses, modules, questions, test cases, and student submissions. Tuned for 500 connections.
- **Redis 7 (e1ab-redis):** Acts as a high-speed cache (for caching generated hints and session data) and as the message broker for Celery.
- **Celery & Celery Beat:**
 - **Worker:** Processes background tasks like invoking the AI LLM for hints asynchronously and generating PDF certificates. Uses prefork pool (--concurrency 32) to scale.
 - **Beat:** Schedules periodic tasks (e.g., auto-advancing students to the next semester every Jan 1 and Jul 1).

2.2 Execution Sandbox

- **Docker Engine:** The underlying runtime for isolating student code.
- **Custom Sandbox Image (e1ab-sandbox):** A custom Linux image preloaded with GCC, OpenJDK 17, CPython 3, and SQLite3.

2.3 AI & Machine Learning (100% Offline)

- **Ollama (e1ab-ollama):** A local LLM server running the qwen2.5-coder:1.5b model. It generates contextual, pure-theory hints for students without sending code to OpenAI/external providers.
- **ChromaDB & Sentence Transformers:** Local Vector Database and embedding models (all-MiniLM-L6-v2) used in the RAG (Retrieval-Augmented Generation) agent to synthesize custom DSA questions offline.

2.4 Frontend & UX

- **Monaco Editor (VS Code core):** Powers the in-browser code editor with syntax highlighting for C, C++, Java, and Python.

- **HTML5 Templates + Vanilla CSS/JS:** Uses Django templates enhanced with "Cyber Violet" styling, micro-animations, glassmorphism, and responsive layouts. No heavy frontend framework (like React) is used to keep the server footprint lean and load times fast.

2.5 Reverse Proxy & Orchestration

- **Nginx:** Sits in front of Gunicorn. Handles gzip compression, static file serving (`/static/` and `/media/`), and proxying API requests to Django.
 - **Gunicorn:** Python WSGI HTTP Server. Configured with 16 workers and 4 threads per worker (`gthread` class) to easily handle 400 concurrent students.
 - **Docker Compose:** Orchestrates the multi-container environment (app, db, redis, celery, nginx, ollama).
-

3. Core Modules Deep Dive

3.1 The Execution Sandbox (`core/sandbox.py`)

When a student submits code, PRAXIS does not run it on the host machine. Instead, it spins up an ephemeral Docker container for sub-second execution.

How it works:

1. **Container Isolation:** The sandbox runs with `--network none` (no internet access), `--cap-drop ALL` (no root capabilities), memory limits (128MB), and CPU limits (0.5 cores).
2. **File Binding:** The system creates a temporary directory in `sandbox_data/`, writes the student's code to it (e.g., `main.c` or resolving the public class name for Java), and bind-mounts it to `/box` in the container.
3. **Base64 Encoding & Execution:** To avoid shell escaping injection vulnerabilities, the python script base64-encodes the source code, passes it into a shell script inside the container (`run_script.sh`), which decodes, compiles, and runs it.
4. **Enforced Time Limits:** Code execution is wrapped in the Linux `timeout` command (e.g., 2.0s). If it exceeds this, the container is killed, and a `Time Limit Exceeded` (TLE) status is returned.
5. **Languages Supported:**
 - **C / C++:** Uses GCC (`-std=c11` , `-std=c++17`). Hard-fails on missing returns or bad main signatures (`-Werror=return-type` , `-Werror=main`).
 - **Java:** Parses the code via Regex to dynamically detect the `public class` name so `javac` doesn't fail on mismatched filenames.
 - **Python:** Direct execution with CPython 3.
 - **SQL:** Combines SQL queries with table schemas and evaluates using `sqlite3` in batch mode.

3.2 Question-Grounded AI Tutor (`core/hint_service.py`)

To assist students without giving away the answers, PRAXIS implements a 3-tier progressive hint system.

How it works:

1. **Zero-Code Policy:** Hints are generated using a local Ollama LLM (`qwen2.5-coder:1.5b`). The system prompt explicitly strictly forbids code syntax, variable declarations, or pseudo-code. It acts as a "Theoretical Computer Science Professor".
2. **Progressive Tiers:**
 - **Tier 1 (Theoretical Foundations):** Core domain theory.
 - **Tier 2 (Algorithmic Paradigm):** Time complexity and logical transitions.

- **Tier 3 (Invariant Analysis):** Edge cases, failures, and invariants.
3. **Asynchronous Architecture:** Web workers (Gunicorn) *never* block waiting for the LLM.
- When a student clicks "Unlock Hint", Django immediately returns a cached **L4 Deterministic Theory Hint** (instant Regex-based fallback rules based on the question topic and compiler errors).
 - Simultaneously, it queues a Celery background task (`generate_llm_hint_task`) to call Ollama. Once Ollama finishes, it overrides the cache with the high-quality LLM response for future requests.
4. **Sanitization (`_sanitize_hint`):** The LLM output is heavily scrubbed using Regex to remove any accidental markdown code blocks or C/Python keywords before being stored in the database.

3.3 Offline Question Generator (RAG) (`core/rag_agent.py`)

Faculty can generate custom, high-quality questions without internet access.

How it works:

1. **Knowledge Base:** PRAXIS holds over 300 curated DSA problems in `data/DSA_Topics/` as markdown files.
2. **Embeddings & ChromaDB:** Using `sentence-transformers`, the markdown files are chunked and embedded into a local ChromaDB vector database.
3. **Retrieval & Adaptation (`generate_question`):**
 - When faculty request a topic (e.g., "Dynamic Programming", "Medium"), the script retrieves the Top 5 semantically similar problems from ChromaDB.
 - A deterministic adaptation script (no LLM required) alters the problem. It adjusts constraints based on the requested difficulty, manipulates the title (e.g., prepending "Optimized" or "Advanced"), formats a new problem description, and injects starter code.
 - It synthesizes new edge-case test combinations from the reference problem and standard generic boundary values.
4. **Result:** A fully formed Django `Question` object is created and bound to the module with 5-8 test cases.

3.4 User-Friendly Proctoring System

PRAXIS enforces fair assessment environments without punishing students for trivial actions.

1. **Debounced Blur:** Standard proctoring systems lock tests immediately if the user switches tabs. PRAXIS debounces the `window.onblur` event (400ms). This forgives accidental taskbar clicks or rapid tab toggling.
2. **Graduated Warnings:** It uses a 3-strike warning system via non-blocking toast notifications. Only on the 3rd strike does a modal halt the test.
3. **In-Editor Flexibility:** Copying and pasting *inside* the Monaco editor is fully permitted, facilitating normal refactoring. It only hooks the global `paste` event to block external clipboard content.

3.5 Certificate Generator (`core/certificate_generator.py`)

When students complete a course with a score above a defined threshold (e.g., 80%):

1. **Evaluation:** A Django management command or Celery task calculates total module scores.
2. **PDF Generation:** Using libraries like `WeasyPrint` or `pydyf` (as defined in requirements), it converts an HTML certificate template into a PDF.
3. **Cryptographic Verification:** A unique UUID signature is generated, embedded into a QR code using the `qrcode` library, and placed on the certificate. Employers can scan the QR code to verify the

certificate's authenticity against the platform's public database.

4. High-Concurrency Tuning

To handle 400 students simultaneously submitting code and unlocking hints:

1. **PostgreSQL:** `max_connections` increased to 500. `shared_buffers` set to 256MB.
2. **Gunicorn:** `16 workers × 4 threads = 64 concurrent threads`.
3. **Celery Preforking:** Worker runs with `--concurrency 32`, able to execute 32 sandbox evaluations or LLM queries in parallel.
4. **Redis Connection Pooling:** Acts as a broker, reducing load on the database for ephemeral states (like hint generation locks).
5. **Stateless Sandbox:** The Docker runtime is completely stateless, meaning multiple containers can spin up and tear down on the same machine without race conditions.

5. Security Summary

- All user-executed code runs inside heavily restricted Docker containers.
- Environment configurations are strictly loaded via `.env`.
- Base64 encoding is used to transfer source code to the container bash script to prevent remote code execution via shell injection on the host.

6. Future Enhancements & Roadmap

While the current architecture robustly supports the existing workload, the following improvements can be made for future scalability and features:

1. **Kubernetes Orchestration:** Migrate from Docker Compose to Kubernetes for autoscaling the Sandbox and LLM execution nodes horizontally across multiple physical servers.
2. **WebSockets for Real-time Feedback:** Transition from polling/REST APIs for sandbox execution and AI hints to WebSockets (via Django Channels). This will reduce HTTP overhead and enable streaming of execution logs and LLM generated tokens in real-time.
3. **Multi-Model LLM Gateway:** Expand the AI tutor to route requests to different models based on complexity. E.g., use a smaller quantized model for basic syntax errors and a larger model for algorithmic hints.
4. **Plagiarism Detection:** Implement an offline AST-based (Abstract Syntax Tree) code similarity checker (e.g., MOSS-like algorithm) to automatically detect potential plagiarism across student submissions.
5. **Advanced Analytics Dashboard:** Add deeper analytics for faculty to identify common conceptual bottlenecks (e.g., tracking which specific topics trigger the highest rate of Tier 3 hint unlocks).
6. **Support for Additional Languages:** Seamlessly integrate Rust, Go, or JavaScript execution into the sandbox to cater to broader placement requirements.